
One Step Forward and K Steps Back: Better Reasoning with Denoising Recursion Models

Chris Cameron¹ Wangzheng Wang^{1,2} Nikita Ivanov^{1,3} Ashmita Bhattacharyya^{1,3} Didier Chételat¹
Yingxue Zhang¹

Abstract

Looped transformers scale computational depth without increasing parameter count by repeatedly applying a shared transformer block and can be used for iterative refinement, where each loop rewrites a full fixed-size prediction in parallel. On difficult problems, such as those that require search-like computation, reaching a highly structured solution starting from noise can require long refinement trajectories. Learning such trajectories is challenging when training specifies only the target solution and provides no supervision over the intermediate refinement path. Diffusion models tackle this issue by corrupting data with varying magnitudes of noise and training the model to reverse it in a *single step*. However, this process misaligns training and testing behaviour. We introduce Denoising Recursion Models, a method that similarly corrupts data with noise but trains the model to reverse the corruption over *multiple* recursive steps. This strategy provides a tractable curriculum of intermediate states, while better aligning training with testing and incentivizing non-greedy, forward-looking generation. Through extensive experiments, we show this approach outperforms the Tiny Recursion Model (TRM) on ARC-AGI, where it recently achieved breakthrough performance. 

1. Introduction

Looped transformers, also known as recursive transformers, are a class of deep learning models that repeatedly apply the same transformer layer to an input to produce a prediction at test time (Giannou et al., 2023). Tying weights across

loops imposes an architectural constraint that encourages the model to learn update rules that generalize across depth, acting as an implicit regularizer that can improve generalization. Empirically, this constraint is associated with stronger algorithmic reasoning (Dehghani et al., 2018; Geiping et al., 2025a) as well as improved parameter efficiency (Saunshi et al., 2025) and sample efficiency (Zhu et al., 2025b).

A novel looped transformer called the Tiny Recursion Model (TRM) (Jolicoeur-Martineau, 2025) recently made headlines by achieving breakthrough performance on ARC-AGI (Chollet et al., 2025), a notoriously difficult general reasoning benchmark consisting of novel puzzles, each with transformation that must be inferred from only a handful of examples. This accomplishment was particularly dramatic as this model, composed of barely seven million parameters, outperformed leading approaches based on Large Language Models (LLMs) such as o3-high (OpenAI, 2025), which has thousands of times more parameters and is pretrained at extreme computational cost.

The TRM is the direct successor of the Hierarchical Reasoning Model (HRM) (Wang et al., 2025) and the latest installment in a line of *iterative-refinement* looped transformers, dating back to Lee et al. (2018). Iterative-refinement models are non-autoregressive: at each loop they predict a complete, fixed-size output in parallel (one token/cell per position). Unlike chain-of-thought which appends newly generated tokens, these models are Markovian as subsequent loops rewrite the same-size prediction. They typically train by starting from noise and applying a sequence of recursions to match the target output. To borrow terminology from the *diffusion* literature, we could call these models *backward-training* as they start from noise and try to undo it, as in the backward process of a diffusion model.

TRMs substantially outperform standard transformers on ARC-AGI-2, however they still leave the majority of ARC-AGI-2 problems unsolved. One explanation is that it is too demanding for the model to discover long, coherent trajectories starting from random noise and ending at highly-structured outputs without any additional supervision. This can be seen as analogous to sparse rewards in reinforcement learning (RL), where a random initial policy rarely

¹Huawei Technologies ²University of Waterloo
³University of Toronto. Correspondence to: Chris Cameron
<chris.cameron@huawei.com>.

stumbles onto a useful path to learn from. Long horizons becomes especially problematic because naively backpropagating through all recursive steps creates training instability (He et al., 2016) and requires storing activations across the entire unrolled computation. In practice, many looped models (Geiping et al., 2025a; Wang et al., 2025), including TRM, therefore rely on *Truncated Backpropagation Through Time* (TBPTT) (Williams and Zipser, 1995); they unroll the recursion for a fixed window of k steps, backpropagate through that window, then detach the state and continue the unrolling.

These long-horizon optimization issues motivate training objectives that avoid backpropagating through long unrolled recursions. Conditional diffusion models (Lai et al., 2025, Chapter 8) provide a clean workaround: they sample intermediate states from a process of corrupting the prediction target and train a *denoiser* model to reverse that corruption in a *single step*. This provides an automatic easy-to-hard curriculum by varying the degree of noise, considerably simplifying the optimization landscape. Perhaps backward-training, without any form of curriculum learning, is simply too demanding on the more difficult problems. Diffusion is also an iterative-refinement model since the same denoiser is applied repeatedly at test time, albeit diffusion has largely developed separately from the looped-transformer literature. Since they start from the target output and add noise to it, we could call these models *forward-training* (in reference to the forward process in diffusion).

While forward-training offers a solution to long-horizon training, we report experimental results in Section 5 showing that it substantially underperforms backward-training on ARC-AGI. We attribute this poor performance to a discrepancy between how the model is trained and how it is used at test time. During training, intermediate states are produced by noising the target and the model is optimized to remove that noise in a *single step*. At test time, however, the model is iterated on its own predictions for many steps; the intermediate states it encounters can therefore differ substantially from the noised-target states seen in training, and the one-step objective does not incentivize learning to produce intermediate states that remain useful under further self-application. Training through a recursive window gives the model the capacity for *implicit planning* (Guez et al., 2019) in latent space before committing to a final answer. This train-test mismatch could lead to compounding errors and encourage greedy updates when the model is unrolled at inference.

In contrast, backward-training methods avoid the train-test discrepancy. This points to a natural goal: incorporate the curriculum learning of forward-training while retaining the train-test alignment of backward-training. We therefore propose Denoising Recursion Models (DRMs), a novel looped

transformer that takes one forward step along the diffusion corruption process into a corrupted target and then learns to recover the clean target over k recursive denoising steps, rather than in a single jump. Crucially, our method only requires optimizing a single recursive window, rather than long episodes with a TBPTT detachment schedule. At test time, prediction proceeds as in TRM and diffusion by repeatedly applying the transformer from a maximally corrupted initialization. See Section 3 for full details.

In extensive experiments on ARC-AGI, using masking as the noise process, we found that DRMs outperformed TRMs at the same parameter count and surpassed the state of the art among open-source LLM-based baselines (NVARC, Sorokin and Puget (2025)) when controlling for training data. In addition, we investigate an alternative diffusion-TRM hybrid approach, which we call the State Perturbation Recursion Model (SPRM), where we inject noise into the intermediate latent states of the TRM procedure during training in order to explore more diverse trajectories. SPRM similarly improves upon TRM in small-data regimes, but it does not match DRM in larger-data regimes. In these data-rich settings, inherent data diversity likely makes the extra exploration from state perturbations unnecessary.

2. Related Work

Recurrent architectures use weight sharing to process sequences or generate computational depth. Recurrent Neural Networks (RNNs) (Elman, 1990) such as LSTMs (Hochreiter and Schmidhuber, 1997) apply recurrence along the sequence dimension, updating a fixed-size hidden state as they step through positions of an input sequence. In contrast, looped transformers apply recurrence along the depth dimension: they process the entire sequence in parallel while iterating a shared layer to scale depth arbitrarily. Dehghani et al. (2018) introduced the autoregressive looped transformer (called Universal Transformer), which is the core architecture behind new looped language models (Geiping et al., 2025b; Zhu et al., 2025b). There is a separate non-autoregressive line of work where a fixed-sized output is recursively updated (Lee et al., 2018).

2.1. Looped Models

By repeatedly applying the same block, looped models can implement sophisticated iterative computations by scaling compute independently of parameter count. This can make them substantially more parameter efficient than non-looped alternatives (Geiping et al., 2025a; Zhu et al., 2025b). At a theoretical level, Giannou et al. (2023) showed that looped transformers can emulate a small instruction-set computer, and subsequent empirical work has demonstrated the practical benefits. Saunshi et al. (2025) showed that a k -layer transformer unrolled for L steps can match the performance

of a non-looped transformer with kL layers. Recurrent parameter sharing may also improve *data efficiency* on algorithmic tasks, since it encourages the model to learn a reusable update rule rather than separate depth-specific computations. Another advantage is that looping supports *latent reasoning*: the model can perform many internal computational steps without externalizing each one as a discrete chain-of-thought token. Merrill and Sabharwal (2025) showed that looped models can represent functions using only logarithmic depth in the number of steps required by discrete chain-of-thought. Finally, looped models often show stronger *generalization to greater problem length or difficulty*, since the same transition function can be unrolled for more steps at test time than during training. Selsam et al. (2018) and Tönshoff et al. (2023) showed that recurrent-depth GNNs trained on SAT generalize to harder instances through deeper unrolling, while Schwarzschild et al. (2021), Bansal et al. (2022), and Fan et al. (2025) showed the same phenomenon in transformer-style models. The model of Bansal et al. (2022) is especially close to the TRM: it is non-autoregressive and encourages loop-invariant computation by applying a progressive loss at a randomly chosen recursion depth, so the model is trained to produce coherent outputs across depths.

Our work represents the latest in a line of research investigating *small* iterative-refinement recursion models, which have demonstrated remarkably data-efficient reasoning without massive pretraining, particularly ARC-AGI. See Appendix A.1 for a detailed overview of ARC-AGI and the most successful solutions. The Hierarchical Reasoning Model (HRM) (Wang et al., 2025) first demonstrated this potential by using alternating “fast” and “slow” recurrent steps, where the fast step is run many times for every execution of the slow step. Despite having only 27M parameters, HRM outperformed high-performance LLMs like o3-mini-high on ARC-AGI. This was simplified by the TRM (Jolicoeur-Martineau, 2025), which uses a single homogeneous recursive layer. Cutting down to just 7M parameters, TRM outperformed HRM and surpassed 100B+ parameter models like o3-high. The Universal Reasoning Model (URM) (Gao et al., 2025) improved TRM by applying a 1D convolution over the token sequences at the end of the MLP block, while Hu et al. (2025) (ViARC) improved over TRM with a larger training dataset, using a similarly small but non-looped transformer with a 2×2 patch encoding/decoding. We believe these improvements of (1) changing encoding/decoding (ViARC) and (2) model architecture (URM) are fundamentally independent of our method and would be easy to integrate into our implementation.

Long-horizon recursive training creates a memory problem since activations scales linearly with unrolled depth and an optimization problem as gradients can degrade. Existing approaches to address these problems include TBPTT for

explicit finite-horizon training (Williams and Zipser, 1995), one-step denoising objectives in diffusion, implicit differentiation in Deep Equilibrium Models (DEQ) (Bai et al., 2019; 2021), and RL-style optimization over long reasoning trajectories in autoregressive and diffusion settings (Black et al., 2024; He et al., 2025). Another related approach applied to non-looped model is to train deep networks one “block” at a time with local denoising objectives (Li et al., 2025; Shing et al., 2026). Earlier layers are trained on more corrupted targets and deeper layers on cleaner ones. The main motivation is more efficient training of deep non-looped models but their method becomes very similar to DRM when the same “block” is recursively applied.

2.2. Diffusion

Diffusion models are a type of looped, iterative-refinement model, first introduced by Sohl-Dickstein et al. (2015). They treat refinement as the reversal of a stochastic corruption process. Ho et al. (2020) introduced Denoising Diffusion Probabilistic Models (DDPM), which is the dominant formalism today. While these foundations were established in continuous state-spaces, they can also be applied to tasks with discrete tokens. One approach is *latent diffusion models* (Rombach et al., 2022) that bridges this gap by mapping discrete tokens into continuous embedding spaces, where standard Gaussian noise is applied and reversed before projecting back to discrete text (Li et al., 2022; Gong et al., 2022). Alternatively, diffusion can operate directly in discrete domains via *stochastic transition matrices* (Austin et al., 2021) where tokens are corrupted by stochastically mapping tokens to a distribution over the vocabulary, or *absorbing masking states* where the models learns to iteratively recover tokens from a masked input (Chang et al., 2022). Recent methods like Llada (Nie et al., 2025; You et al., 2025; Zhu et al., 2025a) have shown that masked diffusion for training language models can be competitive with autoregressive models.

2.3. Intersection of Diffusion and Recursion

Lee et al. (2018) trained an early iterative-refinement model using a *denoising-autoencoder* objective. In our terminology, their procedure stochastically interleaves forward-training (denoising a corrupted version of the target) and backward-training (refining from an uninformative initialization). Although this work predates diffusion terminology, the denoising-autoencoder component plays a similar role to diffusion by providing intermediate difficulty states via controlled corruption of the ground truth. Our DRMs are closely related but differ in how these ingredients are combined. Lee et al.’s forward-training updates supervise a single refinement step, whereas their backward-training updates still require traversing the trajectory from scratch. DRMs instead couple forward corruption with multi-step

refinement: we sample a corruption level, initialize the recursion at the corrupted target, and train to recover the clean output over a window of k recursive steps.

Several recent papers explicitly frame looped models as diffusion-like processes. Geiping et al. (2025b) build on Chen et al. (2024)’s work that proposed an asynchronous update schedule where the model processes token k at loop depth $i - k$, effectively creating a *diagonal frontier*. Unlike our methods, their primary focus is computational efficiency during inference and their experiments use pretrained looped models (Geiping et al., 2025a) that do not use diffusion objectives, whereas our approach is a training method. Similarly, Zhou et al. (2025) characterize looped models as continuous diffusion processes lacking intermediate noise and they propose a hybrid “co-evolutionary” architecture that combines discrete and continuous denoising.

An alternative to standard backward-training methods is to optimize the denoising trajectory with reinforcement learning (RL) (Black et al., 2024). He et al. (2025) recently applied this idea to masked diffusion, using intermediate rewards based on similarity to the ground truth. These rewards are analogous in spirit to the deep-supervision signals used in TRM, but the credit-assignment story is different: because the denoising policy is optimized over the full rollout, each update is valued according to how it contributes to eventual success under the same multi-step process used at inference. In this sense, RL directly aligns the training objective with test-time denoising, and He et al. (2025) report substantial improvements over LLaDA on mathematical and reasoning benchmarks.

A related idea appears in a different RL setting: recurrent policies in external MDPs rather than denoising trajectories. There, repeated internal computation has been discussed as a form of *implicit planning*, where the policy can refine its latent state before committing to an action. Guez et al. (2019) introduced the Deep Repeated ConvLSTM (DRC), and Bush et al. (2025) later provided evidence consistent with planning-like latent updates in this architecture. DRMs train with supervised losses rather than policy optimization, however backpropagating through multiple recursive denoising steps gives the model a related incentive to shape latent states for their downstream usefulness.

3. Methods

3.1. Preliminaries: Training Looped Models

In this work, we operate in the non-autoregressive sequence-to-sequence setting. We are given a dataset of training pairs $(X, Y) \sim \mathcal{D}$, where the input X and ground truth Y are sequences of length M . Both are represented as one-hot encodings over a vocabulary $\mathcal{V}$, such that $X, Y \in \{0, 1\}^{M \times |\mathcal{V}|}$. Each unique token is mapped to a learned

continuous embedding in $\mathbb{R}^d$. To generate predictions from a latent state $H \in \mathbb{R}^{M \times d}$, the model uses a decoder head to generate a probability distribution over the target tokens:

$$P_\theta(Y | H) = \text{softmax}(\text{dec}(H)). \quad (1)$$

We focus on *looped models*, a class of functions where the latent state is generated via a shared transition operator Φ_θ applied recursively:

$$H_t = \Phi_\theta(H_{t-1}, X, \dots), \quad t = 1 \dots T \quad (2)$$

where H_t is the latent state at recursion step t and H_0 is an initialization distribution (typically Gaussian noise). T represents the maximum recursive depth used during inference. While all looped models share this inference structure, they have different ways of training Φ_θ to manage the training instability and memory constraints over a long horizon T .

Many of the methods considered here rely on a forward corruption process $q_\tau(\tilde{Y} | Y)$, instantiated from a generic family $\{q_\tau(\tilde{Y} | Y)\}_{\tau \in [0, 1]}$, where τ denotes the corruption level. This family may represent, for example, Gaussian diffusion for continuous corruption (Ho et al., 2020) or masking for discrete corruption (Nichol and Dhariwal, 2021). We assume only that $\tau \sim \mathcal{U}(0, 1)$, that $q_0(\tilde{Y} | Y)$ leaves the target uncorrupted (i.e., $\tilde{Y} = Y$ with probability 1), and that larger τ corresponds to progressively stronger corruption, with $\tau = 1$ denoting maximal corruption.

3.1.1. FORWARD-TRAINING MODELS (DIFFUSION)

Diffusion models decouple the training procedure from the recursive generation process. A forward process $q_\tau(\tilde{Y} | Y)$ corrupts the ground truth Y into a noisy state $\tilde{Y}_\tau$, effectively fast-forwarding the recursion process to intermediate states of varying levels of completeness. The model is trained to reverse this corruption in a *single step*; Φ_θ is applied exactly once to the embedding of noisy input $\text{Embed}(\tilde{Y}_\tau)$, preventing gradients from flowing through time. Importantly, our objective is a reconstruction objective: the model is trained to predict the clean target Y from $\tilde{Y}_\tau$ (Li and He, 2025), rather than to predict the injected noise or another noised parameterization. We minimize the following reconstruction loss across noise levels τ :

$$\mathcal{L}_{\text{fwd}}(\theta) = \mathbb{E}_\tau[-\log P_\theta(Y | \Phi_\theta(\text{Embed}(\tilde{Y}_\tau), X, \tau))]. \quad (3)$$

3.1.2. BACKWARD-TRAINING MODELS

Backward-training models (like TRM) are trained starting from pure noise and unrolling the recursion. Over long enough horizons, these methods must use some form of *Truncated Backpropagation Through Time* (TBPTT). The common instantiation of this, as is done in TRM, is to partition the full trajectory T into intervals of size k . The

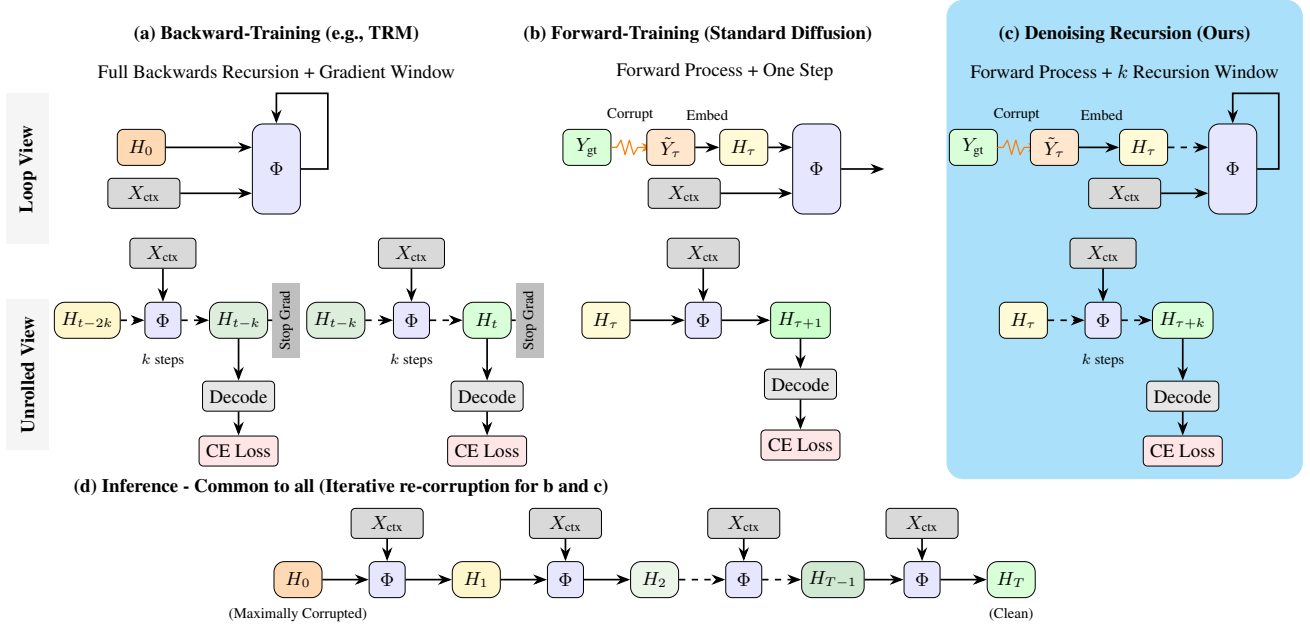


Figure 1. Architectural comparison between (a) backward-training (e.g., TRM), (b) forward-training (Standard Diffusion), and (c) DRMs. Like forward-training, DRMs sample the forward process and like backward-training, they train over k recursions of transition operator Φ . All methods recursively map from maximal corruption to a clean target during inference (d).

gradients are computed within a window, but the recurrent state is detached from the computation graph at the boundaries. Formally, let $sg(\cdot)$ denote the stop-gradient operator. The state update is:

$$\tilde{H}_t = \begin{cases} sg(H_t) & \text{if } t \bmod k = 0 \\ H_t & \text{otherwise} \end{cases} \quad (4)$$

The recursion continues as $H_{t+1} = \Phi_\theta(\tilde{H}_t, X, \dots)$. Supervision is applied at the end of each interval k up to the maximum depth T :

$$\mathcal{L}_{\text{backward}}(\theta) = \sum_{i=1}^{T/k} -\log P_\theta(Y | \tilde{H}_{i \cdot k}). \quad (5)$$

Unlike diffusion, the state $\tilde{H}_{i \cdot k}$ is the result of applying Φ_θ sequentially, allowing the model to learn forward-looking algorithmic operations while maintaining tractable training dynamics via the stop-gradient operator.

3.2. Denoising Recursion Models (DRMs)

DRMs are a hybrid between forward-training and backward-training looped transformers. We leverage the “fast-forward” capability of diffusion to initialize the model at an arbitrary difficulty level (noise scale τ), but we then allow the model to resolve this state over a recursive window of k steps. We initialize the embedded corrupted target:

$H_0 = \text{Embed}(\tilde{Y}_\tau)$ where $\tilde{Y}_\tau$ is sampled from a forward diffusion process $q_\tau(\tilde{Y} | Y)$. We then unroll the transition operator Φ_θ for a fixed window of k iterations and minimize the negative log-likelihood of the clean ground truth Y given the resulting state H_k :

$$\mathcal{L}_{\text{DRM}}(\theta) = \mathbb{E}_\tau [-\log P_\theta(Y | H_k)]. \quad (6)$$

By initializing with $\tilde{Y}_\tau$, we retain the state diversity of diffusion, exposing the model to a vast range of intermediate states without needing to generate them. And by backpropagating through k steps of Φ_θ , we reintroduce the forward-looking inductive bias of recurrent models, training the network to perform multi-step algorithmic refinement rather than greedy denoising.

See Figure 1 for an illustrative comparison. At inference, DRMs initialize from a sample at the maximally corrupted state supported by the forward process and then iteratively apply the denoiser over a sequence of corruption levels $\tau_T > \tau_{T-1} > \dots > \tau_0 = 0$, producing an updated prediction at each step until reaching the uncorrupted state. In the masked discrete case, this corresponds to initializing all output tokens as MASK and repeatedly predicting a full grid followed by partial re-masking.

3.3. State Perturbation Recursion Models (SPRMs)

DRMs start with a forward step, followed by backward steps. We consider an additional hybrid method with *alternating*

backward and forward processes. We call this State Perturbation Recursion Models (SPRMs) because it forces the model to continuously recover from stochastic perturbations along states of its own reasoning path. Rather than denoising from a noisy ground truth, SPRMs effectively denoise from its own incumbent solution. This increases the diversity of intermediate states, which could improve robustness at test time when encountering out-of-distribution states.

Let $q_\tau(\tilde{H} \mid H)$ denote a forward corruption process. At each recursion step t , we sample a noise level τ_t and perturb the previous state via the forward process:

$$\begin{aligned} \tau_t &\sim \mathcal{U}(1, \dots, T), \quad \tilde{H}_{t-1} \sim q_{\tau_t}(\cdot \mid H_{t-1}), \\ H_t &= \Phi_\theta(\tilde{H}_{t-1}, X). \end{aligned} \quad (7)$$

This corruption forces the operator Φ_θ to map from the noisy latent manifold back to the solution manifold at every iteration. We think continuous state perturbations could help learn a robust attractor field around correct reasoning trajectories and avoid converging to brittle trajectories.

4. Experimental Setup

4.1. Data

We evaluate on two datasets: ARC2-Public Eval (120 difficult tasks) and ARC2-Public Train, which we refer to as *ARC-Easy* (1000 generally easier tasks). For pretraining, we use three settings of increasing scale: ARC-Easy + ConceptARC (Moskvichev et al., 2023), reARC (Hodel, 2024), and NVARC-Train (Sorokin and Puget, 2025). We exclude NVARC-Eval because it is derived from ARC2-Public Eval and would introduce data leakage. Dataset sizes are summarized in Table 1. Each ARC task provides a small set of input-output training examples and 1–3 held-out test inputs, where each grid is at most 30×30 and each cell takes one of 10 colours. See Appendix F.8 for an evaluation on Sudoku and Maze tasks evaluated by Jolicoeur-Martineau (2025).

4.2. Model Architecture

The methods in Section 3 are agnostic to the particular architecture used to instantiate them. In experiments, we fix a single concrete model instantiation across all methods: we use the TRM architecture and codebase of Jolicoeur-Martineau (2025) for the backward-training model, and we build DRM, SPRM, and all relevant baselines within this same implementation. Our comparison does not vary the transition operator Φ_θ and the entire surrounding architectural and training scaffold inherited from TRM. Aside from modifications that are intrinsic to each training method, all components and hyperparameters are held constant. See Table 3 in the appendix for the full hyperparameter details.

The TRM instantiation includes several components that

were omitted from the abstract formulation in Section 3. First, the model uses a learned **task embedding**, concatenated at each recursion step, to share information across examples from the same task. Second, the recurrent state is organized as a **scratchpad** (y_t, z_t) , where y_t is mapped to output tokens and z_t is an unconstrained latent workspace for computation. Third, the TRM training recipe includes **warm-up recursion steps** before gradients are applied. Finally, the model includes a scalar **Q-head** that predicts exact-match correctness and is used for early exit and for selecting a final solution across augmentations. See Figure 7a in the appendix for pseudocode. These components are retained across our model variants unless a change is required by the method definition itself.

4.3. DRM Corruption Process Instantiation

Given that we tokenize each cell in an ARC grid, we instantiate $q_\tau(\tilde{Y} \mid Y)$ as a *discrete masking schedule*—an established technique in language diffusion (Nie et al., 2025)—rather than adding Gaussian noise to continuous embeddings¹. We used a cosine noise schedule, observing faster training convergence compared to a linear or sigmoid schedule (see Appendix F.4). This aligns with prior work on discrete diffusion (Chang et al., 2022) and continuous diffusion (Nichol and Dhariwal, 2021). Linear schedules tend to destroy information too often, while cosine schedules smooth out the rate of information destruction. Let $\tau \in [0, 1]$ represent the normalized diffusion timestep. We compute the signal retention rate $\bar{\alpha}_\tau$ as: $\bar{\alpha}_\tau = \cos^2\left(\frac{\pi\tau}{2}\right)$, where the fraction of tokens to be masked is $r(\tau) = 1 - \bar{\alpha}_\tau$. For a diffusion timestep $\tau \sim \mathcal{U}[0, 1]$, we selected $\lfloor r(\tau) \cdot M \rfloor$ indices uniformly at random; this creates a “salt and pepper” masking that we thought could be quite easy to interpolate, however we did not observe any benefit from sampling in a spatially correlated way by using a 2D Gaussian kernel where full “objects” were more likely to be masked. We replaced the tokens at those indices with a special MASK token and used separate token embeddings for mapping the masked ground truth into latent space than those used for input grids. Since TRM adds the input embedding to the output embedding, we thought these separate embeddings would help to avoid interference. See Figure 7b for pseudocode.

During inference, we reverse this process using an iterative “generate-and-remask” strategy as in Nie et al. (2025) except we do not freeze tokens in place if they are not selected to be remasked. We define a sequence of discrete timesteps $s_T, \dots, s_0$ decreasing from 1 to 0—for every example, we sample uniformly between 1 and 0 and sort in descending order. We initialize the grid as fully masked. At each itera-

¹We first tried standard DDPM on the grid embeddings but observed poor performance, likely because the model strategically learned embeddings that were recoverable from noise.

tion i , the model predicts logits for all grid cells given the current input. We first decode a fully populated candidate grid $\hat{y}_0$ by taking the argmax of the logits at every position. To construct the input for the subsequent step $i-1$, we calculate the required number of masked tokens $j = \lfloor r_{s_{i-1}} \cdot M \rfloor$ based on the schedule. We then select j indices *uniformly at random* to be re-masked as in training, retaining the predicted values from $\hat{y}_0$ at the unselected indices. This process of predicting the full grid and stochastically remasking a subset of cells, repeats until $s_0 = 0$, at which point no masks remain. We tried confidence-based remasking—i.e., masking lowest confidence tokens (Nie et al., 2025)—but did not observe any performance improvement. See Appendix B for examples of DRM inference.

4.4. SPRM Corruption Process Instantiation

For our secondary hybrid method, we inject noise directly into the hidden states rather than the input tokens. This perturbation is applied exclusively at the truncation points (“outer loops” of TRM). We instantiate $q_\tau(\tilde{Y} \mid Y)$ with the standard linear variance schedule from Denoising Diffusion Probabilistic Models (DDPM) (Ho et al., 2020). We define a variance schedule β_t that increases linearly from $\beta_{\text{start}} = 10^{-4}$ to $\beta_{\text{end}} = 0.02$. At the end of each truncated backpropagation window (every k steps), we perturb the hidden state $H_{i:k}$ before passing it to the next window. We use the variance-preserving update: $H_{\text{next}} = \sqrt{1 - \beta_\tau} H_{\text{prev}} + \sqrt{\beta_\tau} \epsilon$, $\epsilon \sim \mathcal{N}(\mu(H_{\text{prev}}), \Sigma(H_{\text{prev}}))$, $\tau \sim \mathcal{U}(1 \dots T)$. The noise is scaled with the mean and variance of the latent state H_{prev} . We do not add noise during inference, since we observed minor performance reduction when we tried.

4.5. Baselines

Standard Diffusion and Transformer Baselines. To isolate the necessity of the recursive inductive bias, we evaluated standard architectures that match the computational budget of the TRM but lack the recurrent weight sharing. We evaluated standard diffusion models using the exact same discrete masking schedule and inference procedure as the DRM implementation described in Section 4.3. The 7M model matches the capacity of the DRM transition operator. The 70M model is constructed by stacking transformer blocks to match the total computational depth (FLOPs) used by the TRM during a single backpropagation window (which consists of four recurrent steps plus one final processing loop). We also evaluated a standard Transformer with the same stacked architecture. To determine if recursive layers are required specifically within a gradient block, we also evaluated the repeated application of a stacked transformer.

TRM without deep supervision. We include a TRM baseline where we trained solely on the loss at the final step of the recursion window, removing the intermediate trun-

cated backpropagation losses. This serves to measure the dependence of the architecture on dense supervision signals.

State-of-the-Art (NVARC). We compare our results against the current state-of-the-art (among open-source models) NVARC system (Sorokin and Puget, 2025), which achieved first place in the 2025 ARC Challenge. We evaluated their 4-billion parameter LLM while controlling for the dataset.

Other baselines We also reproduced the ViARC (Hu et al., 2025) results from the reARC pretraining setting. We observed severe optimization instability when pretraining on NVARC data so we do not include those results. We were not able to benchmark URM (Gao et al., 2025) because of persistent training crashes.

4.6. Training and Evaluation

We follow the standard TRM protocol of Jolicoeur-Martineau (2025) unless specified otherwise. We did all exploratory hyperparameter tuning on ARC2-Easy to avoid overfitting ARC2-Eval. See the appendix for specific training (Table 2) and architecture (Table 3) parameters. See our official GitHub repository [here](#).

Data Preparation and Augmentation. For each ARC task, we sample 1000 augmentations. For each augmentation we (1) randomly permute the colours, (2) select a random transformation (e.g., flip, rotate) and (3) if the grid is smaller than 30×30 , randomly translate each pair of grids into a 30×30 template (cells outside the original are masked when loss is measured). We flatten grids in row-major order with learned colour/MASK embeddings and RoPE (Su et al., 2024).

Evaluation Protocol. We follow the released TRM implementation exactly. Although the TRM paper describes ARC evaluation as selecting the most common prediction over 1000 test-time augmentations, the public code additionally aggregates predictions across evaluation snapshots during finetuning (Huang et al., 2017). We used this protocol for a direct comparison to TRM. In our setup, evaluations were performed every 10K epochs, so the final candidate pool contains predictions from all saved finetuning checkpoints and all 1000 augmentations. After undoing the augmentations, identical predictions are grouped across this full pool and primarily by vote count, with mean predicted q used as a tie-breaker—Table 1 reports pass@2 exact-match accuracy. In our own ablations, evaluating only the final checkpoint tended to give slightly worse results.

Pretraining and Finetuning. For the large-scale data settings, we used a two-stage process: we pretrained the model on reARC for 100K epochs and on NVARC+reARC for 2K epochs, using identical hardware for both. Following pretraining, we performed a finetuning stage of 300K epochs, where we strictly followed the TRM methodology of jointly training on the provided training pairs of both ARC2-Train

and ARC2-Eval tasks. We save checkpoints every 10K finetuning epochs, and final ARC2-Eval predictions are obtained by aggregating candidates from all saved checkpoints using the q -value ranking procedure described above, rather than evaluating only the terminal checkpoint.

NVARC Baseline Details. We pretrained NVARC until the validation loss plateaued (approx. 2K epochs). We used Sorokin and Puget (2025)’s test-time training implementation for finetuning to ARC2-Easy and ARC2-Eval.

5. Results

The Necessity of Recursion. We first establish the baseline difficulty of the task. We find that standard non-recursive architectures are insufficient for ARC-AGI. A standard transformer achieved only 25.5% on ARC-Easy, far below the recursive TRM baseline of 45.7%. Crucially, we found that diffusion without backpropagating through multiple recursive steps performs very poorly. A standard diffusion model (matching the DRM architecture but without looping) failed completely, achieving $\sim 0\%$ accuracy. Even when we stacked layers to match the compute budget of recursion (70M params), we still could not get above 1%. This suggests that iterative refinement via shared weights is the primary driver of performance.

DRM Stabilizes Training without Deep Supervision. Standard TRMs rely heavily on deep supervision (TBPTT) to guide intermediate states. When we removed this supervision (“TRM w/o deep sup”), performance fell by 11% (from 45.7% to 34.7%) on ARC-Easy. In contrast, our DRM method trains without any intermediate gradient truncation or deep supervision yet outperformed the TRM baseline by nearly 5% on ARC-Easy (50.5% vs 45.7%). This suggests that the diffusion objective provides a useful curriculum for learning long-horizon reasoning trajectories.

DRM Outperforms TRM. These gains transfer to the ARC2-Eval benchmark, which consists of tasks with higher difficulty. On the base setting (no pretraining), DRM boosted ARC2-Eval performance from 6.3%² to 9.6% and the performance of TRM without deep supervision was just 3.3%. Even in the high-data regime (NVARC pretraining), DRM improved ARC2-Eval accuracy from 12.5% to 16.7% (7M params) and 21.8% to 24.9% (14M params).

Performance Scaled with Data and Model Size. We observe a strong synergistic effect between model and dataset size. Scaling from the 7 million parameter baseline (and four recurrent gradients steps) to the 14 million parameter (and six recurrent gradients steps) improved performance 3–4% when restricted to pretraining on ARC2-Train. The

increased capacity from a bigger model became more critical when leveraging massive pretraining datasets. Adding NVARC to pretraining improved 7M DRM by approximately 7% but with 14 million parameters, this boosted performance by 11.5%. This indicates that the smaller model is capacity constrained, whereas the 14M architecture can absorb the high diversity of the 47K tasks from NVARC.

DRM Outperformed SOTA 4B LLM Method. Our 14M-parameter DRM outperformed the current state-of-the-art NVARC (Qwen3-4B Base) model (24.9% vs 22.6%) when trained on the same data. This result is particularly striking because NVARC is a highly engineered system built around a 4-billion parameter model, specialized specifically for ARC-AGI. This demonstrated that DRM (and TRM) can be orders of magnitude more parameter efficient (14M vs. 4B).

State Perturbation Recursion Model. Our SPRM method also improved over the TRM baseline (50.7% vs. 45.7% on ARC2-Train and 6.3% vs. 10.1% on ARC2-Eval). However, its effectiveness was mixed in other settings. On the reARC dataset, SPRM performed slightly worse than the TRM baseline. We hypothesize that the high diversity of reARC (1000 examples per task) saturated the model’s need for exploration, making noise injection redundant.

6. Discussion

ARC vs. Other Settings where Diffusion Excels. On ARC, discrete masked diffusion baselines fail almost completely even when scaled to match the TRM compute budget. ARC is a discriminative point-to-point mapping rather than to a distribution-to-distribution generative setting where diffusion usually excels. In image generation, denoising can often progress from coarse global structure to fine local detail, and many outputs may be acceptable. ARC has a single exact target, and many tasks require globally coordinated discrete transformations where locally plausible partial states are incompatible with the final solution. This mismatch may help explain why standard masked diffusion performs poorly even though diffusion is highly effective in conventional generative domains.

Importance of Backpropagating Through Recursion. While DRMs improve over TRMs, the difference in performance is much more modest than the gap between TRM and diffusion baselines. These findings suggest that the core ingredient to the success of these small models is backpropagating through a window of recursions. We see two complementary mechanisms. First, this provides *multi-step credit assignment*: intermediate states are trained to be useful because they enable progress several steps later, which discourages greedy updates that could be very poor for certain tasks—e.g., in obstacle navigation, a move that decreases Manhattan distance can still result in a dead end. Recursive

²Jolicoeur-Martineau (2025) reported 7.8% in their paper with the identical setup and codebase.

Table 1. Pass@2 exact-match accuracy results on held-out data for every task comparing **DRM** and **SPRM** methods against the TRM baseline and state-of-the-art NVARC. The 7M models use identical architectures. The 14M settings increased embedding size from 512 to 768, number of heads from 8 to 12, and gradient loops k from 4 to 6. *indicates significant improvement over TRM (permutation test, $p < .05$) with identical model architecture (p -values in Appendix F.2).

Method	#Params	ARC-Easy	ARC2-Eval (by Pretraining Data)		
		(No Pretrain)	+ARC-Easy	+reARC	+NVARC Train
		-	1k tasks (3.5 ex/task)	400 tasks (1k ex/task)	47k tasks (24 ex/task)
Baselines					
Transformer	7M	10.9	-	-	-
Transformer	70M	25.5	1.7	-	-
Transformer <i>w/ deep sup</i>	70M	37.2	2.5	-	-
Masked Diffusion (Standard)	7M	0.0	0.0	-	-
Masked Diffusion (Standard)	70M	0.7	0.0	-	-
TRM <i>w/o deep sup</i>	7M	34.7	3.3	-	-
TRM <i>w/o inner loop</i>	34M	39.8	3.0	-	-
ViARC	18M	-	-	8.3	-
LLM (NVARC)	4B	43.2	13.8	15.6	22.6
TRM (Baseline)	7M	45.7	6.3	12.4	12.5
SPRM (Ours)		50.7*	10.1*	11.8	14.4
DRM (Ours)		50.5*	9.6*	14.7	16.7
TRM (Baseline)	14M	53.2	10.6	18.8	21.8
SPRM (Ours)		55.4*	11.8	17.4	22.2
DRM (Ours)		55.0*	13.3	21.0	24.9

denoising creates conditions under which planning-like latent computation (Guez et al., 2019) could emerge. Second, weight tying across depth can act as a strong regularizer for algorithmic reasoning. Because the same parameters operate across many latent states and depths, gradients from different unrolled steps must largely agree for an update to persist; spurious, step-specific correlations are more likely to generate conflicting signals that cancel, biasing learning toward stable, reusable computations.

DRM Balances “off-policy” and “on-policy” Learning. DRM improves upon TRM in part by mitigating long-horizon training instability. Pure backward-training is effectively “on-policy”: the model must discover long trajectories from noise using only its own self-generated intermediate states, which are often uninformative early in training. DRM initializes training from corrupted targets, providing an “off-policy” curriculum of intermediate states closer to the solution. From that starting point onward, training is “on-policy” where each later state is generated by the model itself rather than supplied by the corruption process.

Scaling Behaviour. The strong interaction between model size, unrolled gradient depth, and pretraining scale suggest that recursion models are not limited to the “tiny” regime, rather their parameter efficiency can be converted into improved performance as data and compute increase.

7. Conclusions

We introduced the Denoising Recursion Model (DRM), a looped transformer that resolves the opposing limitations of (1) backward-training methods that struggle with long-horizon stability and (2) diffusion that suffers from a myopic objective and a misalignment between training (on noised target) and testing (on self-generated trajectories). We effectively combine the training objectives: we corrupt the target with noise identical to the diffusion process, but train the model to recover the clean signal over a sequence of recursive steps, rather than a single jump. Our method consistently outperformed the TRM baseline on ARC-AGI across a variety of settings. On the largest pretraining dataset, we achieved 24.9% accuracy on ARC2-Eval, surpassing the winner of the recent ARC Challenge (NVARC) when trained on identical data. We also showed large improvements from scaling the architecture from 7M to 14M parameters, suggesting that further scaling could yield substantial gains. At present, progress on ARC-AGI is difficult to interpret because gains from model architecture and training objectives are often entangled with gains from more sophisticated synthetic-data generation. Our results suggest that substantial headroom remains in the learning algorithm itself, and motivate evaluations that make these sources of progress easier to disentangle.

Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Akyürek, E., Damani, M., Zweiger, A., Qiu, L., Guo, H., Pari, J., Kim, Y., and Andreas, J. (2024). The surprising effectiveness of test-time training for few-shot learning. [arXiv preprint arXiv:2411.07279](#).
- Austin, J., Johnson, D. D., Ho, J., Tarlow, D., and Van Den Berg, R. (2021). Structured denoising diffusion models in discrete state-spaces. [Advances in neural information processing systems](#), 34:17981–17993.
- Bai, S., Kolter, J. Z., and Koltun, V. (2019). Deep equilibrium models. [Advances in neural information processing systems](#), 32.
- Bai, S., Koltun, V., and Kolter, J. Z. (2021). Stabilizing equilibrium models by jacobian regularization. [arXiv preprint arXiv:2106.14342](#).
- Bansal, A., Schwarzschild, A., Borgnia, E., Emam, Z., Huang, F., Goldblum, M., and Goldstein, T. (2022). End-to-end algorithm synthesis with recurrent networks: Extrapolation without overthinking. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, [Advances in Neural Information Processing Systems](#), volume 35, pages 20232–20242. Curran Associates, Inc.
- Black, K., Janner, M., Du, Y., Kostrikov, I., and Levine, S. (2024). Training diffusion models with reinforcement learning. In [The Twelfth International Conference on Learning Representations](#).
- Bush, T., Chung, S., Anwar, U., Garriga-Alonso, A., and Krueger, D. (2025). Interpreting emergent planning in model-free reinforcement learning. In [The Thirteenth International Conference on Learning Representations](#).
- Chang, H., Zhang, H., Jiang, L., Liu, C., and Freeman, W. T. (2022). Maskgit: Masked generative image transformer. In [Proceedings of the IEEE/CVF conference on computer vision and pattern recognition](#), pages 11315–11325.
- Chen, B., Martí Monsó, D., Du, Y., Simchowitz, M., Tedrake, R., and Sitzmann, V. (2024). Diffusion forcing: Next-token prediction meets full-sequence diffusion. [Advances in Neural Information Processing Systems](#), 37:24081–24125.
- Chollet, F. (2019). On the measure of intelligence. [arXiv preprint arXiv:1911.01547](#).
- Chollet, F., Knoop, M., Kamradt, G., Landers, B., and Pinkard, H. (2025). Arc-agi-2: A new challenge for frontier ai reasoning systems. [arXiv preprint arXiv:2505.11831](#).
- Chung, J., Kastner, K., Dinh, L., Goel, K., Courville, A. C., and Bengio, Y. (2015). A recurrent latent variable model for sequential data. [Advances in neural information processing systems](#), 28.
- Cole, J. and Osman, M. (2025). Don’t throw the baby out with the bathwater: How and why deep learning for arc. [arXiv preprint arXiv:2506.14276](#).
- Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, Ł. (2018). Universal transformers. [arXiv preprint arXiv:1807.03819](#).
- Dieng, A. B., Ranganath, R., Altosaar, J., and Blei, D. (2018). Noisin: Unbiased regularization for recurrent neural networks. In [International Conference on Machine Learning](#), pages 1252–1261. PMLR.
- Elman, J. L. (1990). Finding structure in time. [Cognitive science](#), 14(2):179–211.
- Fan, Y., Du, Y., Ramchandran, K., and Lee, K. (2025). Looped transformers for length generalization.
- Franzen, D., Disselhoff, J., and Hartmann, D. (2025a). The architects - technical report: Arc prize 2025 solution summary. Technical report, JGU Mainz and Lambda, Inc.
- Franzen, D., Disselhoff, J., and Hartmann, D. (2025b). Product of experts with llms: Boosting performance on arc is a matter of perspective. [arXiv preprint arXiv:2505.07859](#).
- Gao, Z., Chen, L., Xiao, Y., Xing, H., Tao, R., Luo, H., Zhou, J., and Dai, B. (2025). Universal reasoning model. [arXiv preprint arXiv:2512.14693](#).
- Geiping, J., McLeish, S., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., and Goldstein, T. (2025a). Scaling up test-time compute with latent reasoning: A recurrent depth approach. [arXiv preprint arXiv:2502.05171](#).
- Geiping, J., Yang, X., and Su, G. (2025b). Efficient parallel samplers for recurrent-depth models and their connection to diffusion language models. [arXiv preprint arXiv:2510.14961](#).
- Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. (2023). Looped transformers as programmable computers. In [International Conference on Machine Learning](#), pages 11398–11442. PMLR.

- Gong, S., Li, M., Feng, J., Wu, Z., and Kong, L. (2022). Diffuseq: Sequence to sequence text generation with diffusion models. [arXiv preprint arXiv:2210.08933](#).
- Google DeepMind (2026). Gemini deep think. Large language model.
- Guez, A., Mirza, M., Gregor, K., Kabra, R., Racanière, S., Weber, T., Raposo, D., Santoro, A., Orseau, L., Eccles, T., et al. (2019). An investigation of model-free planning. In [International conference on machine learning](#), pages 2464–2473. PMLR.
- He, H., Renz, K., Cao, Y., and Geiger, A. (2025). Mdp0: Overcoming the training-inference divide of masked diffusion language models. [arXiv preprint arXiv:2508.13148](#).
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In [Proceedings of the IEEE conference on computer vision and pattern recognition](#), pages 770–778.
- Ho, J., Jain, A., and Abbeel, P. (2020). Denoising diffusion probabilistic models. [Advances in neural information processing systems](#), 33:6840–6851.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. [Neural computation](#), 9(8):1735–1780.
- Hodel, M. (2024). Addressing the abstraction and reasoning corpus via procedural example generation. [arXiv preprint arXiv:2404.07353](#).
- Hu, K., Cy, A., Qiu, L., Ding, X. D., Wang, R., Zhu, Y. E., Andreas, J., and He, K. (2025). Arc is a vision problem! [arXiv preprint arXiv:2511.14761](#).
- Huang, G., Li, Y., Pleiss, G., Liu, Z., Hopcroft, J. E., and Weinberger, K. Q. (2017). Snapshot ensembles: Train 1, get m for free. In [5th International Conference on Learning Representations, ICLR 2017](#).
- Jolicoeur-Martineau, A. (2025). Less is more: Recursive reasoning with tiny networks. [arXiv preprint arXiv:2510.04871](#).
- Lai, C.-H., Song, Y., Kim, D., Mitsufuji, Y., and Ermon, S. (2025). The principles of diffusion models. [arXiv preprint arXiv:2510.21890](#).
- Lee, J., Mansimov, E., and Cho, K. (2018). Deterministic non-autoregressive neural sequence modeling by iterative refinement. [arXiv preprint arXiv:1802.06901](#).
- Li, Q., Teh, Y. W., and Pascanu, R. (2025). Noprop: Training neural networks without full back-propagation or full forward-propagation. In [Proceedings of the 4th Conference on Lifelong Learning Agents](#). PMLR.
- Li, T. and He, K. (2025). Back to basics: Let denoising generative models denoise. [arXiv preprint arXiv:2511.13720](#).
- Li, X., Thickstun, J., Gulrajani, I., Liang, P. S., and Hashimoto, T. B. (2022). Diffusion-lm improves controllable text generation. [Advances in neural information processing systems](#), 35:4328–4343.
- Lim, S. H., Erichson, N. B., Hodgkinson, L., and Mahoney, M. W. (2021). Noisy recurrent neural networks. [Advances in Neural Information Processing Systems](#), 34:5124–5137.
- Merrill, W. and Sabharwal, A. (2025). A little depth goes a long way: The expressive power of log-depth transformers. [arXiv preprint arXiv:2503.03961](#).
- Moskvichev, A., Odouard, V. V., and Mitchell, M. (2023). The conceptarc benchmark: Evaluating understanding and generalization in the arc domain.
- Nichol, A. Q. and Dhariwal, P. (2021). Improved denoising diffusion probabilistic models. In [International conference on machine learning](#), pages 8162–8171. PMLR.
- Nie, S., Zhu, F., You, Z., Zhang, X., Ou, J., Hu, J., ZHOU, J., Lin, Y., Wen, J.-R., and Li, C. (2025). Large language diffusion models. In [The Thirty-ninth Annual Conference on Neural Information Processing Systems](#).
- OpenAI (2025). Introducing OpenAI o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>.
- OpenAI (2026). Gpt-5.4 pro xhigh. Large language model.
- PoetiQ (2025). PoetiQ meta-system and arc-agi solver. https://poetiQ.ai/posts/arcagi_verified/. Software wrapper. Accessed: 2026-04-17.
- PoetiQ Team (2025). Traversing the frontier of superintelligence. PoetiQ AI Blog. Accessed: 2026-01-26.
- Pourcel, J., Colas, C., and Oudeyer, P.-Y. (2025). Self-improving language models for evolutionary program synthesis: A case study on arc-agi. [arXiv preprint arXiv:2507.14172](#).
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In [2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition \(CVPR\)](#), pages 10674–10685. IEEE.

- Saunshi, N., Dikkala, N., Li, Z., Kumar, S., and Reddi, S. J. (2025). Reasoning with latent thoughts: On the power of looped transformers. [arXiv preprint arXiv:2502.17416](#).
- Schwarzschild, A., Borgnia, E., Gupta, A., Huang, F., Vishkin, U., Goldblum, M., and Goldstein, T. (2021). Can you learn an algorithm? generalizing from easy to hard problems with recurrent networks. [Advances in Neural Information Processing Systems](#), 34:6695–6706.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. (2018). Learning a sat solver from single-bit supervision. [arXiv preprint arXiv:1802.03685](#).
- Shing, M., Koyama, M., and Akiba, T. (2026). Diffusionblocks: Block-wise neural network training via diffusion interpretation. In [The Fourteenth International Conference on Learning Representations](#).
- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., and Ganguli, S. (2015). Deep unsupervised learning using nonequilibrium thermodynamics. In [International conference on machine learning](#), pages 2256–2265. PMLR.
- Sorokin, I. and Puget, J.-F. (2025). NVARC solution to ARC-AGI-2 2025. Kaggle ARC Prize 2025, 1st Place Solution. Also available at <https://github.com/lytic/NVARC>.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. (2024). Roformer: Enhanced transformer with rotary position embedding. [Neurocomputing](#), 568:127063.
- Tönshoff, J., Kisin, B., Lindner, J., and Grohe, M. (2023). One model, any csp: Graph neural networks as fast global search heuristics for constraint satisfaction. In Elkind, E., editor, [Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI-23](#), pages 4280–4288. International Joint Conferences on Artificial Intelligence Organization. Main Track.
- Wang, G., Li, J., Sun, Y., Chen, X., Liu, C., Wu, Y., Lu, M., Song, S., and Yadkori, Y. A. (2025). Hierarchical reasoning model. [arXiv preprint arXiv:2506.21734](#).
- Williams, R. J. and Zipser, D. (1995). Gradient-based learning algorithms for recurrent networks and their computational complexity. In [Backpropagation: theory, architectures, and applications](#), pages 433–486.
- Xu, Y., Khalil, E. B., and Sanner, S. (2023). Graphs, constraints, and search for the abstraction and reasoning corpus. In [Proceedings of the AAAI Conference on Artificial Intelligence](#), volume 37, pages 4115–4122.
- You, Z., Nie, S., Zhang, X., Hu, J., Zhou, J., Lu, Z., Wen, J.-R., and Li, C. (2025). Llada-v: Large language diffusion models with visual instruction tuning. [arXiv preprint arXiv:2505.16933](#).
- Zhou, C., Yang, C., Hu, Y., Wang, C., Zhang, C., Zhang, M., Mackey, L., Jaakkola, T., Bates, S., and Zhang, D. (2025). Coevolutionary continuous discrete diffusion: Make your diffusion language model a latent reasoner.
- Zhu, F., Wang, R., Nie, S., Zhang, X., Wu, C., Hu, J., Zhou, J., Chen, J., Lin, Y., Wen, J.-R., et al. (2025a). Llada 1.5: Variance-reduced preference optimization for large language diffusion models. [arXiv preprint arXiv:2505.19223](#).
- Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., Li, L., Shi, J., Ma, K., Li, S., Kergan, T., Smith, A., Qu, X., Hui, M., Wu, B., Min, Q., Huang, H., Zhou, X., Ye, W., Liu, J., Yang, J., Shi, Y., Lin, C., Zhao, E., Cai, T., Zhang, G., Huang, W., Bengio, Y., and Eshraghian, J. (2025b). Scaling latent reasoning via looped language models.

A. Background

A.1. ARC-AGI

While modern AI systems excel at tasks where massive training data is available, they struggle with test-time adaptation—the ability to acquire new skills on the fly during deployment. Accurately evaluating this adaptability is difficult: given the scale of modern pretraining corpora, it is nearly impossible to verify whether a benchmark task is truly novel to a model or simply recalled from memory. To address this challenge, [Chollet \(2019\)](#) introduced the Abstraction and Reasoning Corpus (ARC-AGI): a diverse set of novel tasks where each task provides a few-shot context (2–5 input-output grid pairs). See [Figure 2](#) for an example.

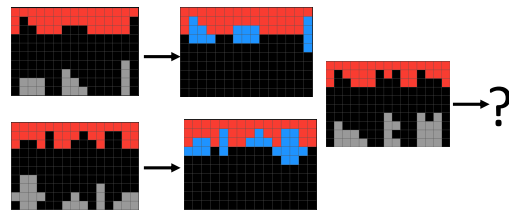


Figure 2. Example of ARC-AGI task. Two input-output grids examples for the task transformation: move each gray blocks in the input into the corresponding slots in the output and change colours to blue. Must apply the transformation correctly to a held-out input to be correct.

Each task is defined by a singular, simplest latent algorithm that uniquely explains the transformation across the examples. Solving a task requires reverse-engineering this algorithm to determine the exact output grid for an unseen

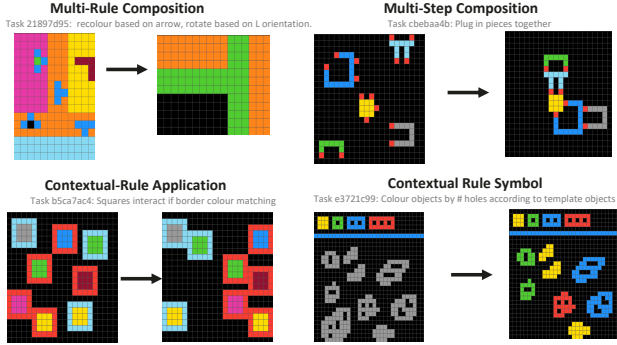


Figure 3. Various examples of the flavours of reasoning needed to solved ARC-AGI-2 tasks.

test input. Since each task is unique and shares only abstract “core knowledge priors” (e.g., objectness, symmetry), models cannot rely on retrieving pre-learned solutions. See Figure 3 for diverse examples of ARC-AGI-2 tasks. Instead, they must perform explicit new learning to induce the transformation rule. ARC-AGI remains a formidable challenge for AI, serving as a primary leaderboard for progress toward general intelligence (Chollet et al., 2025).

While some early work attempted to solve ARC-AGI using discrete search (Xu et al., 2023), neural methods now dominate the literature. These approaches can be broadly categorized into *program synthesis* and *direct grid prediction*. A program synthesis approach is currently state of the art on ARC-AGI-2. Poeti Team (2025) leveraged top-tier closed-source LLMs to generate code solutions mapping input to output grid and use iterative prompting strategies to verify and refine solutions. Code generation tends to be not nearly as advantageous with open-source models, though Pourcel et al. (2025) recently achieved competitive results by constructing a large dataset reinterpreting its self-generated incorrect programs as correct programs for new tasks.

Our work primarily compares to open-source *direct grid prediction* methods, which treat ARC as a token or image generation problem. The standard recipe for these models involves three components: (1) pretraining on large-scale synthetic datasets, (2) test-time fine-tuning on augmentations of the few-shot examples, and (3) search-based decoding with selection mechanisms like majority voting (Cole and Osman, 2025; Akyürek et al., 2024; Franzen et al., 2025b). The winner of the 2025 ARC Challenge (Chollet et al., 2025), NVARC (Sorokin and Puget, 2025), followed this formula using Franzen et al. (2025b)’s approach, with the critical innovation being a clever mechanism to leverage LLMs to generate a massive, diverse synthetic dataset for pretraining.

The second-place solution (Franzen et al., 2025a) is particularly relevant as it adapts Llada (Nie et al., 2025), a

pre-trained diffusion language model. However, their approach differs fundamentally from ours. They rely on a standard, fixed-depth transformer architecture with 4 billion parameters and repeatedly re-masking and denoising at test time. In contrast, our method uses diffusion to train a recursive, parameter-efficient loop from scratch.

Beyond architecture, recursion models differ from these LLM-based approaches in how they process few-shot examples. LLM approaches rely on *in-context learning*, where all input-output pairs are concatenated into a single prompt. In contrast, HRM, TRM, and URM all train a shared *task embedding* with the few-shot examples. The model then conditions on this embedding to process the test input. Conceptually, the forward pass in an LLM must simultaneously infer the transformation and apply it; in these models, the forward pass can be viewed as “executing” the learned task program (the embedding) on the provided input.

Although our method outperformed the strongest commercial LLMs available in November 2025, the frontier has advanced rapidly since then. Recent systems such as Gemini 3 Deep Think (Google DeepMind, 2026) and GPT-5.4 Pro (xHigh) (OpenAI, 2026) exceed 80% on this benchmark, far above our best result of 25%. Direct comparison should be interpreted cautiously: the architectures and training pipelines of these systems are not public, but they surely use orders of magnitude more data and compute than our models, and may also incorporate benchmark-targeted synthetic data. In addition, agentic systems such as Poeti’s (Poeti, 2025) suggest that coding-based search and verification loops are especially effective for ARC-AGI. We suspect that such mechanisms play an important role in these recent gains. We evaluated our model at just 7M or 14M parameters, where learning general coding capabilities is likely out of reach.

B. DRM Reasoning Examples

We now show a few examples of DRM execution on test examples from ARC-AGI-2 Eval. During inference, DRM takes 16 denoising steps. We show the sequential predictions plus the remaskings for each step according to our noise schedule. We show the input and ground truth label at top of the image and the internal confidence $\hat{P}(\text{correct})$ across steps in the top right. Each prediction grid is annotated with the internal confidence and each remasking grid is annotated with remasking ratio—the fraction of cells being remasked.

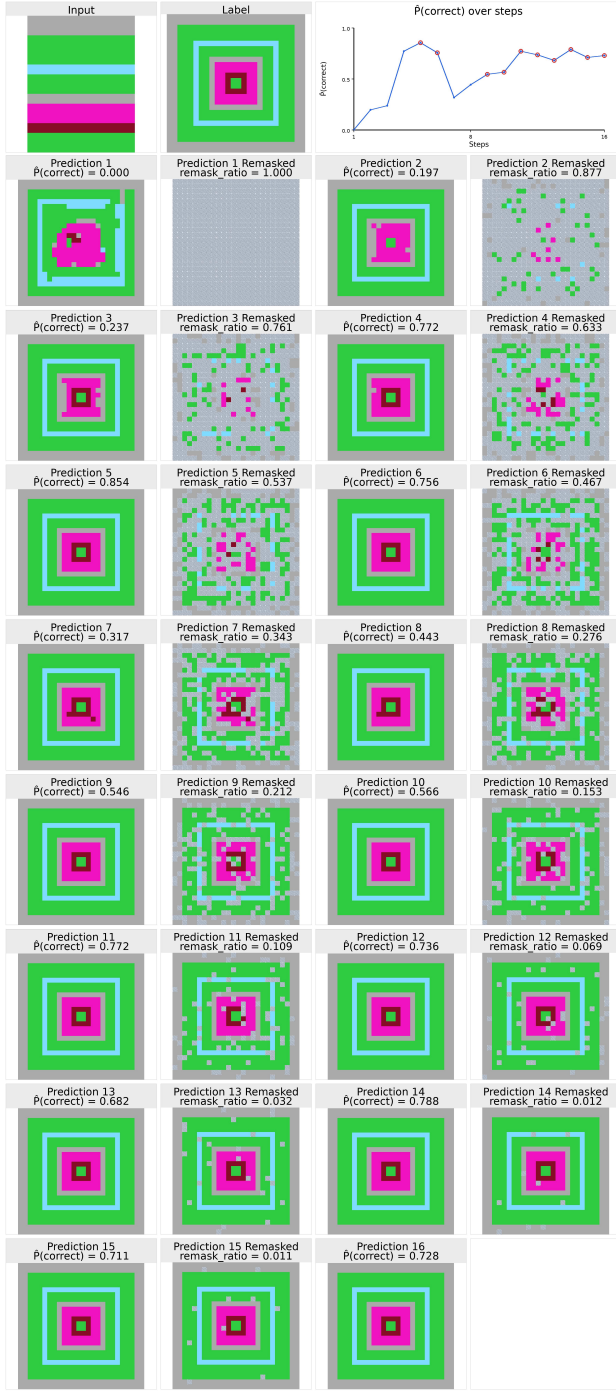


Figure 4. Successful convergence on the ARC-AGI 2 task [45a5af55](#) of constructing nested squares with a certain colour order specified in the input. The model initially generates an incorrect square with a rough approximation of the correct answer and it refines it over several steps and eventually converges to the correct answer. Self-confidence $\hat{P}(\text{correct})$ is highly correlated with correctness indicating the model understood when it was correct and incorrect.



Figure 5. Successful solution on the ARC-AGI 2 task 2d0172a1 of identifying core shapes from a rough “sketch”. Model is initially confused but eventually makes progress before oscillating between correct and incorrect solution and happens to settle on the correct answer in the final step.



Figure 6. Failure to converge to a correct answer on a ARC-AGI task 8b7bacbf to fill in shapes corresponding to what colour path they lie on. The model seems to understand that the internals of the shapes must be filled but it clearly does not know what the colouring rule is. However, the model shows very low internal confidence indicating it knows it is confused.

C. Pseudocode

See Figure 7 for pseudocode comparing the training logic between TRM and DRM.

D. DRM/TRM relative strengths and weaknesses

Overall, we found the tasks which are solved by each method to be highly correlated. We find that both are strongest on tasks where the transformation may be complex to uncover but easy to apply (e.g., fill in symmetry behind an occlusion: a6f40cea) but both struggle on tasks with transformations requiring significant test-time search (e.g., plugging pieces together in the correct arrangement: cbebaa4b). To understand where each method’s blind spots were relative to one another, we collected all tasks for each method that was solved at least once over the 1000 augmentations (pass@1000) on the largest pretraining scenario with 14M parameters.

Here are the 16 tasks that only DRM solved: 0934a4d8, 2ba387bc, 2c181942, 2d0172a1, 4c3d4a41, 4c416de3, 4c7dc4dd, 64efde09, 8e5c0c38, a395ee82, a6f40cea, b0039139, b99e7126, b9e38dc0, c7f57c3e, dbff022c

We can imagine for many of these how a curriculum could help. For example, in task 2c181942 (<https://arcprize.org/tasks/2c181942>), four different objects need to be placed in the correct positions. With masking, DRM will see many types of partial solutions of one, two, and three objects being correctly placed and perhaps makes it more manageable to discover the correct transformation.

Here are the 8 tasks that only TRM solved: 1ae2feb7, 2b83f449, a251c730, a47bf94d, bf45cf4b, da515329, dd6b8c4b, edb79dae

In Appendix F.8, we show experiments where TRM outperformed DRM at Sudoku. The tasks where TRM is better make sense in light of this. Similar to Sudoku, 5/8 tasks require an unusual degree of test-time search i.e., even if you knew the transformation (rules of the task), it requires some thinking to apply. For example, task dd6b8c4b (<https://arcprize.org/tasks/dd6b8c4b>) requires tracing each of the brown pixels to see if there is a viable path to the centre region. Because DRM is not trained over very long TBPTT sequences, it may not learn to build reasoning over very long reasoning traces as is required in tasks like this.

E. Training Details

See Table 2 for training details and Table 3 for architecture details.

F. Extended Results

F.1. Pass@k Results

Figure 8 shows pass@k ARC2-Eval results for the 14M evaluations shown in Table 1. When pretrained on the largest NVARC dataset, DRM shows a clear improvement up to

(a) TRM (Deep Supervision)

```

def trm_rec(x, y, z, n, T):
    with torch.no_grad():
        for j in range(T-1):
            for i in range(n):
                z = net(x + y + z)
            y = net(y + z)
        for i in range(n):
            z = net(x + y + z)
        y = net(y + z)
    return (y, z), dec(y), Q(y)

for x, y_true in train_data:
    x = input_embed(x)
    # random initialization
    y, z = y_rand, z_rand
    # iter over recursion windows
    for step in num_windows:
        (y, z), y_hat, q_hat = trm_rec(x, y, z)
        loss = ce(y_hat, y_true)
        loss += bce(q_hat, (y_hat==y_true))
        loss.backward()
        opt.step()
        # truncated backpropagation
        opt.zero_grad()
        # early-stopping
        if q_hat > 0:
            break
    
```

(b) DRM (Ours)

```

def drl_rec(x, y, z, n, T):
    with torch.no_grad():
        for j in range(T-1):
            for i in range(n):
                z = net(x + y + z)
            y = net(y + z)
        for i in range(n):
            z = net(x + y + z)
        y = net(y + z)
    return (y, z), dec(y), Q(y)

for x, y_true in train_data:
    x = input_embed(x)
    z = z_rand
    # noise target
    tau = sample_time_step()
    yn = add_noise(y_true, tau)
    yn = label_embed(yn)
    (y, z), y_hat, q_hat = drl_rec(x, yn, z)
    loss = ce(y_hat, y_true)
    loss += bce(q_hat, (y_hat==y_true))
    loss.backward()
    opt.step()
    
```

Figure 7. Comparison of training logic between TRM and DRM (with TRM base). Text in red highlights logic that only appears in TRM, text in green highlights logic that only appears in DRM. (a) TRM uses truncated backpropagation. (b) Denoising Recursion uses a noise curriculum without truncated backpropagation.

pass@1000, where it is $\sim 5\%$ better than the TRM baseline. However, when pretraining on ARC2-train, TRM matches DRM performance at pass@100 and pass@1000. The gap between pass@1 and pass@1000 shows the potential for further improvement. We believe that generation of the correct output is much more difficult than identifying the correct output over a candidate set—i.e., we hypothesize humans would easily discriminate over correct and incorrect outputs.

F.2. Significance Results

To test whether DRM and SPRM significantly improved over TRM, we performed a permutation test over the tasks under the null hypothesis of no performance difference. See Table 4 for p-values. The statistical power is much higher on ARC-Easy than ARC2-Eval given it contains 1000 as opposed to 120 tasks.

F.3. Scaling with gradient loops and warm-up loops

We investigated the effect of varying the number of recursive steps within a gradient window k on the 7M ARC-Easy setting for both TRM and DRM. Following the TRM archi-

tecture, both methods perform two warm-up iterations with no gradient steps per loop prior to the k gradient-bearing recursions. See Table 5 for results.

Both methods substantially improve with increasing k . DRM suffers more at small k , with TRM outperforming DRM at $k = 1, 2$, the two methods reaching approximate parity at $k = 3$, and DRM surpassing TRM at $k = 4$. At $k = 8, 12$, we trained for double the number of epochs since training progress is much slower. At $k = 12$, TRM outperformed DRM. DRM convergence is much slower at this larger k . While TRM performance had plateaued at termination, DRM was consistently increasing. We expect that the DRM/TRM gap would be similarly big if we trained until convergence. Further study is needed to understand why DRM is converging slower for this higher k .

We note that standard masked diffusion corresponds to DRM with $k = 1$ and no warm-up loops. The fact that DRM at $k = 1$ achieves 31.8% accuracy—compared to approximately 0% for standard diffusion—indicates that the warm-up iterations, despite carrying no gradients, play a meaningful role in initializing the model’s hidden state. For $k = 1$, we show below the dependence of DRM and TRM on these

Hyperparameter	Train-from-scratch	Pretrain		Finetune
		reARC	NVARC	
Epochs	100k	100k	2k	300k
LR	1e-4	1e-4	1e-4	1e-4
Puzzle_emb_lr	1e-2	1e-2	1e-2	1e-2
lr_warmup_steps	2k	10k	2k	2k
lr_anneal	no	no	no	no
Batch size	768	768	768	768
Weight decay	0.1	0.1	0.1	0.1
EMA	True	True	True	True
Loss	CE	CE	CE	CE
Optimizer	AdamW	AdamW	AdamW	AdamW

Table 2. TRM training hyperparameters for each stage.

Hyperparameter	TRM/SPRM/DRM		TRM (loop untie)	Transformer
	7M	14M		
Layers	2	2	2	10
Warm-up Windows (H cycles)	3	3	3	1
Gradient Loops (L cycles)	4	6	4(untied)	1
Hidden size	512	768	512	768
Num heads	8	12	8	12
Expansion	4	4	4	4
Max halt steps	16	16	16	16

Table 3. Model/config hyperparameters across methods.

warm-up steps. Both methods show very poor performance with no warmup loops but DRM is especially affected. We also looked at the impact warm-up steps with deeper recursions. At $k=12$, TRM achieved 57.0% and DRM achieved just 38.8%. We suspect that the warm-up loops allow better test-train alignment because the gradient loop starts with a self-generated intermediate state rather than starting directly from the masked target. We suspect TRM is less affected because it passes the latent state forward during TBPTT. Understanding the relative contributions of gradient loops and warm-up loops is an important direction of study.

F.4. Noise Schedule and Mask Sampler Ablation

We ablate the choice of noise schedule and mask sampler on the 7M ARC-Easy setting. We consider three noise schedules—linear, sigmoid, and cosine—paired with two mask sampling strategies: uniform random sampling and spatially correlated sampling using a 2D Gaussian kernel. The linear schedule weights all masking fractions equally, the sigmoid schedule heavily biases toward extreme (very high or very low) fractions, and the cosine schedule represents a middle ground with moderate bias toward both extremes. Using uniform masking, the cosine schedule achieved 50.5% accuracy, followed by linear at 46.2% and sigmoid at 43.6%.

This is consistent with findings in prior diffusion work suggesting that a moderate bias toward high and low masking fractions is beneficial. We also compared uniform and spatially correlated masking under the cosine schedule; uniform achieved 50.5% while spatially correlated achieved 49.0%.

F.5. TRM Extreme Robustness to Overfitting

Figure 9 illustrates an unusual training dynamic: validation continues to improve well after the model has effectively “solved” the training set. In particular, once training exceeds 99% per-pixel exact accuracy across the entire output grid (100k steps), validation metrics keep rising from 38% to 45% between 100k and 500k steps. Note that here we are training on ARC2-train with $\sim 3k$ examples. Since our batch size is 768, four steps is virtually one epoch. This means that TRM does not overfit even after seeing an example about 100k times! Note that we do rotate through 1000 different invertible “augmentations” per data point (e.g., colour permutation, rotation), which does some of the work to avoid overfitting to basic artifacts.

When we instead stack layers without looping, the generalization is much worse. Figure 10 shows that while training loss converges much faster (left) when we stack (Trans-

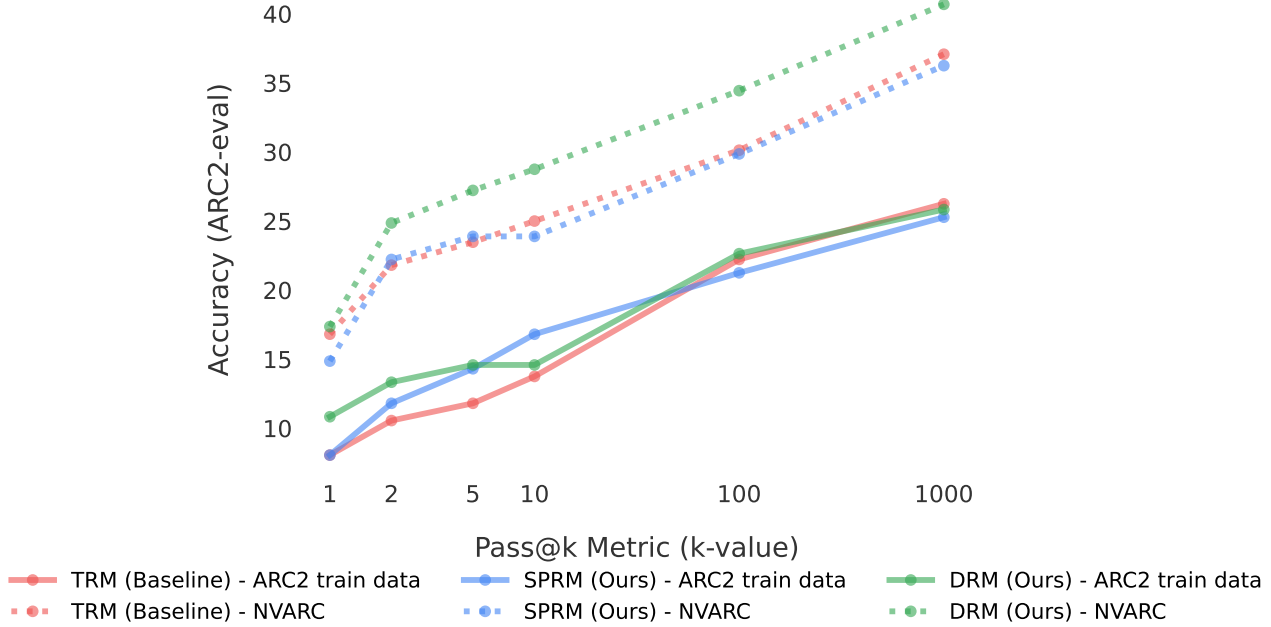


Figure 8. Pass@k exact-match accuracy on ARC2-Eval for three methods (TRM baseline, SPRM, and DRM) with 14M parameters and pretrained with two scales of data (ARC2-train and NVARC)

Test	# params	ARC-Easy	ARC2-eval	+reARC	+NVARC
SPRM>TRM	7m	4.53×10^{-6}	0.0205	0.632	0.202
DRM>TRM	7m	4.65×10^{-5}	0.0313	0.143	0.0772
SPRM>TRM	14m	0.0449	0.266	0.744	0.500
DRM>TRM	14m	0.0418	0.124	0.251	0.183

Table 4. p -values for permutation tests given the null hypothesis that TRM is better.

Table 5. Pass@2 exact match-accuracy on ARC-Easy across k Values

Model	k=1	k=2	k=3	k=4	k=8	k=12
DRM	31.8	37.8	45.1	50.5	55.7	45.3
TRM	37.2	41.0	44.5	45.7	53.7	56.4

Table 6. Pass@2 exact match-accuracy on ARC-Easy across warm-up steps with $k=1$.

Model	no warm-up	1 warm-up	2 warm-up
DRM	0.0 (standard diffusion)	17.7	31.8
TRM	11.0	33.6	37.2

Table 7. Pass@2 exact match-accuracy across various DRM masking noise schedules.

Noise Schedule	ARC-Easy
Sigmoid	43.6
Linear	46.2
Cosine	50.5
Cosine (spatially correlated)	49.0

former), exact-match training performance of the looped model (TRM) quickly surpasses the stacked model (centre). This suggests that “global” understanding is clearly worse when stacking. This translates to much better validation performance (right).

E.6. SPRM is a good regularization of input

SPRM injects noise into the latent state that is carried over and used as the input to the next refinement step. We view noise injection as a regularizer, promoting generalization by

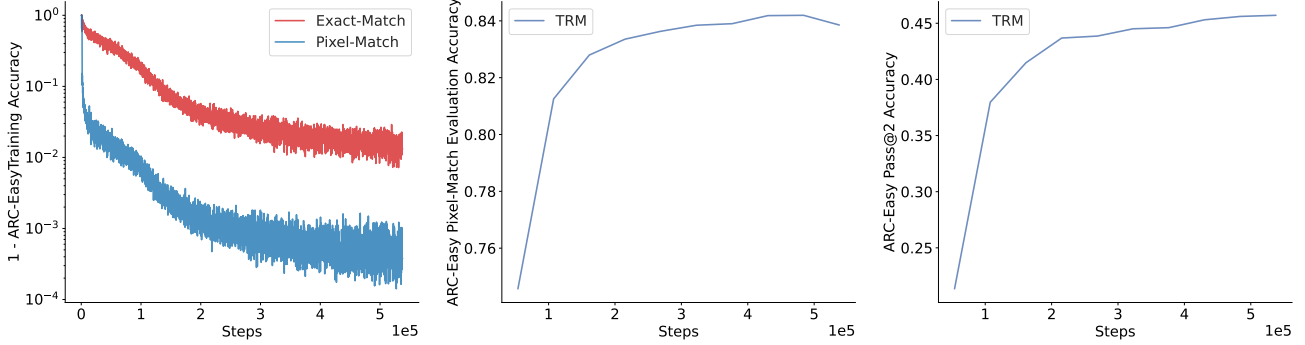


Figure 9. **Extreme robustness to overfitting.** TRM validation accuracy continues to improve even after training surpasses 99% per-pixel accuracy across the grid. (left) 1-exact-match accuracy, 1-per-cell accuracy, (middle) validation per-cell accuracy, (right) validation pass@2.

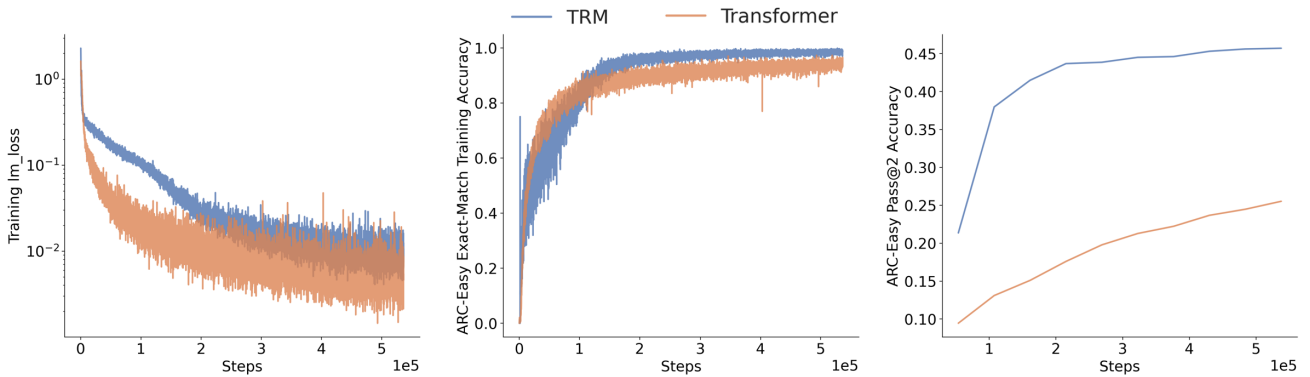


Figure 10. Unlooped transformer has better training loss than TRM baseline (left)) but worse training exact-match accuracy (centre) and higher evaluation pass@2 accuracy (right).

increasing the diversity of training states because the same underlying example can generate many nearby latent inputs across refinement steps.

Figure 11 further indicates that SPRM outperform TRM from an optimization perspective. Adding noise actually improves convergence during training. By “thickening” the refinement trajectory, it may smooth the effective loss landscape since gradients are averaged over latent perturbations rather than tied to one precise hidden trajectory. It is counter-intuitive how adding noise could improve training loss and accuracy but this does align with prior findings on RNNs that injecting noise into intermediate recurrent states can promote a contractive, stable recursion (Lim et al., 2021). We also note that noising intermediate steps is similar to Variational RNNs (Chung et al., 2015) which introduce latent random variables into the recurrence. Prior work on RNNs showed that noise injection can outperform Dropout (Dieng et al., 2018).

F.7. Latent “scratchpad” is not needed

HRM (Wang et al., 2025) maintains two latent states: a low-level state z_L that repeatedly processes the input signal,

and a high-level state z_H that acts as a slower, summarizing carrier across cycles. z_L is essentially a scratchpad state that continually is carried forward while z_H is decoded into a solution. TRM adopts a similar two-latent-state design but simplifies the architecture by tying the high-level update to the same network used for low-level transitions. The HRM pattern:

$$z_L \leftarrow f_L(z_L, z_H + x), \quad z_H \leftarrow f_H(z_H, z_L)$$

becomes an update where both transitions reuse a single module f :

$$z_L \leftarrow f(z_L, z_H + x), \quad z_H \leftarrow f(z_H, z_L). \quad (8)$$

A natural question is whether this scratchpad z_L is truly necessary. In Jolicoeur-Martineau (2025)’s ablations, they test a more aggressive simplification that only depends on z_H

$$z_H \leftarrow f(z_H, x), \quad z_H \leftarrow f(z_H).$$

which performed substantially worse. In contrast, our ablation shows that the scratchpad z_L is not needed if we get rid

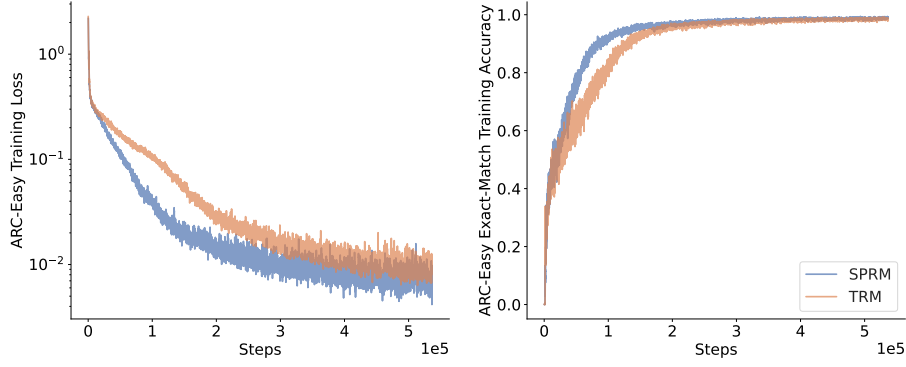


Figure 11. Training dynamics comparison between TRM and SPRM. (left) training exact accuracy and (right) training loss.

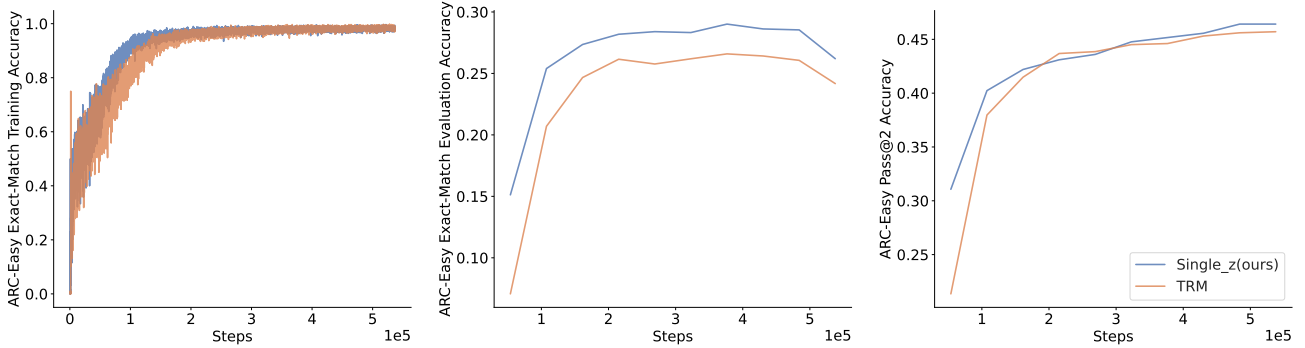


Figure 12. Single_z (no scratchpad latent state) performed as good as the TRM baseline.

of this “summarization” step:

$$z_H^{(t+1)} \leftarrow f(z_H^{(t)}, x).$$

Pass@2 exact-match accuracy remains on par with the baseline as shown in Figure 12 on ARC2-Easy. This suggests that the performance drop observed in their ablation is not due to removing z_L , but rather due to introducing an extra latent-only transition $z_H \leftarrow f_H(z_H)$.

This simplification was also made by the contemporaneous URM (Gao et al., 2025) and we argue this is cleaner since the recursion becomes perfectly homogeneous across iterations. We do notice that removing the scratch pad resulted in better training accuracy yet on-par validation accuracy. This perhaps points to a regularization effect of the scratch pad latent. The transition function f in Equation 8 must learn whether to update z_L or z_H depending on the input, which constrains the model and perhaps makes it more difficult to overfit.

In our experiments, we chose to use the identical scratchpad latent in DRM and TRM to isolate the impact of our training method.

E.8. Maze and Sudoku

To assess the generality of DRM beyond ARC-AGI, we evaluated on the Maze and Sudoku benchmarks studied by Jolicoeur-Martineau (2025) (Table 8). We used the identical architecture to the 7M parameter setting used for ARC experiments and we used identical training and inference compute for DRM and TRM.

Table 8. Pass@2 exact match-accuracy

Model	Maze	Sudoku
TRM	78%	87.9%
DRM	81%	20%
Hybrid (DRM + TBPTT)	—	85.7%

On the Maze task, DRM—where we mask only path tokens—achieved 81% accuracy compared to 78% for TRM. We note that this evaluation likely underestimates the true performance of both methods, as we identified several instances in which TRM and DRM produced valid maze paths that differed from the ground-truth solution. Since the current evaluation protocol requires an exact match to the reference path, these correct but alternative solutions are scored as failures.

On Sudoku, DRM achieved 20% accuracy compared to

87.9% for TRM. We attribute this gap to the qualitative difference between Sudoku and typical ARC tasks. Sudoku puzzles are characterized by rules that are straightforward to understand but solutions that require extensive test-time search. Empirically, we observed that when TRM training on ARC converges, the average number of gradient windows required to solve training problems is just above one (1.2), whereas on Sudoku this number increases to approximately four. Because DRM does not train with truncated backpropagation through time (TBPTT) over multiple gradient windows, it may not learn to reason effectively over the very long sequential dependencies that Sudoku demands.

To further investigate this hypothesis, we conducted an additional experiment in which we corrupt the target as in DRM but then truncate and carry forward as in TRM. This hybrid approach achieved 85.7% accuracy (compared to 87.9% for standard TRM), suggesting that the carry-forward mechanism has importance for Sudoku that it does not have on ARC. This result indicates training DRM over longer gradient windows may help to close the performance gap.

When interpreting these results, it is important to consider that a 20% vs 80% gap on ARC corresponds to solving four times as many unique tasks, whereas on Sudoku it corresponds to solving four times as many instances of effectively the same task. We observed a sudden phase transition during TRM training on Sudoku, where accuracy jumped from approximately 20% to 80%. This behavior is consistent with the hypothesis that the Sudoku benchmark is best viewed as a single task of roughly uniform difficulty: once the model crosses a capability threshold, performance increases simultaneously across all instances. Unlike ARC, where many heterogeneous tasks produce a smoothly increasing performance curve, the uniform nature of Sudoku may cause the performance gap between DRM and TRM to appear larger than the underlying capability difference.